

Module 2 — Spring Core (IoC, DI, Bean Lifecycle & the Container)

This is the highest-priority module. Everything in Spring is a bean living in an **ApplicationContext**. Interviewers want to hear *how the container works internally* — not just “@Autowired injects dependencies.”

2.1 Inversion of Control (IoC) & Dependency Injection (DI)

Why Interviewers Ask This

It's the foundational idea. Coming from Node (where you `require()` and `new` things yourself), you must articulate why handing object creation to a container improves testability, decoupling, and lifecycle management.

Core Concept

- **IoC** — you don't create/wire your dependencies; the *container* does. Control of object graph construction is *inverted* to the framework.
- **DI** — the mechanism: dependencies are *injected* (constructor, setter, field) rather than looked up. Program to interfaces; the container supplies impls.

Internal Working

The container reads bean definitions (from annotations/@Configuration/XML), builds a **BeanDefinition** registry, resolves dependency graph order, instantiates singletons eagerly at startup, and injects collaborators. Backed by **DefaultListableBeanFactory**.

ASCII — IoC Container

```
@Configuration / @ComponentScan
|
BeanDefinitionReader ---> [ BeanDefinition registry ]
|
BeanFactoryPostProcessors (modify definitions, e.g. @Value placeholders)
|
instantiate + inject (resolve graph) ---> singleton cache
|
BeanPostProcessors (wrap: AOP proxies, @Async, @Transactional)
|
ApplicationContext (ready)
```

Real Production Example

A **PaymentService** depends on a **PaymentGateway** interface. Prod wires **StripeGateway**; tests inject a **MockGateway**. No code change — just different beans/profiles. That swap-ability is the payoff of DI.

Advantages / Trade-offs

- Loose coupling, testability (inject mocks), centralized lifecycle, AOP hooks. – Startup magic can be hard to trace; indirection; slower cold start; runtime (not compile-time) wiring errors if misconfigured.

Common Mistakes

Doing `new` inside beans (defeats DI); calling `context.getBean()` manually (service locator anti-pattern); mixing business logic with wiring.

Interview Q / Follow-ups

- IoC vs DI — are they the same? (*IoC is the principle; DI is one implementation.*)

- Types of DI and which is best? (*constructor* — see 2.5.)
- `BeanFactory` vs `ApplicationContext`? (*ApplicationContext = BeanFactory + i18n, events, AOP, auto post-processors, eager singletons.*)

Hands-on Exercise

Define a `NotificationService` interface with `EmailImpl` and `SmsImpl`; inject the desired one via `@Qualifier`/`@Primary`.

Best Practices

Program to interfaces; constructor injection; never use the container as a service locator in business code.

2.2 Bean Lifecycle

Why Interviewers Ask This

Reveals depth: initialization order, `BeanPostProcessor`, and where proxies are applied (critical for understanding `@Transactional`/`@Async`).

Core Concept & Execution Flow

```
1. Instantiate (constructor)
2. Populate properties / inject dependencies
3. Aware callbacks: BeanNameAware, BeanFactoryAware, ApplicationContextAware
4. BeanPostProcessor.postProcessBeforeInitialization
5. @PostConstruct
6. InitializingBean.afterPropertiesSet() / @Bean(initMethod=...)
7. BeanPostProcessor.postProcessAfterInitialization <-- AOP proxy created here
8. Bean is READY (in use)
   ... container shutdown ...
9. @PreDestroy
10. DisposableBean.destroy() / @Bean(destroyMethod=...)
```

Internal Working

`BeanPostProcessor` is the extension point: Spring's own `AbstractAutoProxyCreator` wraps beans in AOP proxies during *after* initialization. That's why a self-invocation (`this.method()`) bypasses `@Transactional`/`@Async` — it doesn't go through the proxy.

ASCII — Where the Proxy Wraps

```
raw bean --BPP.after--> [ Proxy (Tx/Async advice) ] --> injected everywhere
caller -> proxy.method() -> advice (begin tx) -> target.method() -> commit
BUT: target.this.other() bypasses proxy => no advice!
```

Real Production Example

`@PostConstruct` warms a cache after DI completes; `@PreDestroy` gracefully drains a connection pool on shutdown.

Common Mistakes / Debugging

Relying on field-injected deps inside the constructor (not injected yet); heavy work in constructors; self-invocation breaking transactions; expecting `@PreDestroy` on prototype beans (Spring doesn't manage prototype destruction).

Interview Q / Follow-ups

- Full bean lifecycle order.
- `@PostConstruct` vs `InitializingBean` vs `@Bean(initMethod=...)`.
- Why does self-invocation break `@Transactional`? (*proxy bypass* — see AOP.)
- What is a `BeanPostProcessor` vs `BeanFactoryPostProcessor`? (*BFPP edits definitions before instantiation; BPP edits bean instances.*)

Hands-on Exercise

Implement a bean with `@PostConstruct`, `InitializingBean`, and a custom `BeanPostProcessor` logging each phase; observe the order.

Best Practices

Prefer `@PostConstruct`/`@PreDestroy`; keep constructors cheap; never rely on self-invocation for AOP behavior — call through an injected reference or split into another bean.

2.3 Bean Scopes

Scope	Meaning	Use
singleton (default)	one instance per container	stateless services
prototype	new instance per <code>getBean</code> /injection	stateful/short-lived
request	one per HTTP request	web only
session	one per HTTP session	web only
application	one per <code>ServletContext</code>	web only
websocket	one per WebSocket session	web only

Gotcha — singleton depends on prototype: the prototype is injected *once* at singleton creation, so you don't get a fresh instance per call. Fix with `@Lookup` method injection, `ObjectProvider<T>`, or a `Provider<T>`.

Interview Q

- Are Spring singletons the same as GoF singletons? (*No — one per container, not per JVM; and not `private` constructors.*)
- Are singleton beans thread-safe? (*Only if stateless — you must ensure it.*)
- How to inject a prototype into a singleton correctly?

2.4 Stereotype Annotations & Configuration

`@Component` is the generic stereotype; the others are specializations picked up by component scanning:

Annotation	Layer	Extra behavior
<code>@Component</code>	generic	base for the rest
<code>@Service</code>	business logic	semantic only
<code>@Repository</code>	persistence	exception translation (<code>PersistenceExceptionTranslationPostProcessor</code> → <code>DataAccessException</code>)
<code>@Controller</code>	web MVC	view-returning handlers
<code>@RestController</code>	web REST	<code>@Controller</code> + <code>@ResponseBody</code>
<code>@Configuration</code>	config	CGLIB-proxied so <code>@Bean</code> methods return singletons
<code>@Bean</code>	factory method	programmatic bean registration (3rd-party types)

@Configuration internal trick

@Configuration classes are enhanced by CGLIB so that inter-bean method calls (`beanA()` calling `beanB()`) return the *shared singleton*, not a new instance. With `@Configuration(proxyBeanMethods=false)` (Spring Boot's "lite" mode) that interception is skipped for speed — then each call makes a new object.

@Bean VS @Component

- @Component — Spring instantiates the class you own (component scan).
- @Bean — you write a factory method; use for third-party classes you can't annotate (e.g. `ObjectMapper`, `RestTemplate`, a `DataSource`).

Interview Q / Follow-ups

- @Component vs @Service vs @Repository — real differences? (mostly semantic; @Repository adds exception translation.)
- Why is @Configuration proxied? What is `proxyBeanMethods=false`?
- When @Bean over @Component?

2.5 Component Scan, Autowiring & Injection Types

Component Scan

`@ComponentScan(basePackages=...)` (implicit via `@SpringBootApplication` on the main package) discovers stereotyped classes and registers `BeanDefinition`s. Filters: `includeFilters/excludeFilters`.

Autowiring resolution order

1. **By type** (the default for a single candidate).
2. If multiple candidates: `@Primary` wins, else `@Qualifier("name")`, else match by field/param **name**, else `NoUniqueBeanDefinitionException`.
3. No candidate + `required=true` → `NoSuchBeanDefinitionException` (use `ObjectProvider/`
`@Autowired(required=false)/Optional<T>` for optional).

Constructor vs Field vs Setter Injection

	Constructor	Field	Setter
Immutability	<code>final</code>		
Required deps enforced			
Testable without Spring	(just <code>new</code>)	(reflection)	partial
Circular deps	fails fast (good)	silently allowed	allowed
Recommended	Yes	No	optional deps

Since Spring 4.3, a single constructor needs no `@Autowired`. Field injection is discouraged (hidden deps, hard to test, can't be `final`).

ASCII — Autowiring Decision

```

need bean of type T
|-- 1 candidate -----> inject it
|-- N candidates:
|   @Primary present? -----> inject primary
|   @Qualifier given? -----> inject named
|   name matches param? -----> inject by name
|   else -----> NoUniqueBeanDefinitionException
|-- 0 candidates -----> NoSuchBeanDefinitionException (unless optional)

```

Interview Q / Follow-ups

- Constructor vs field injection — why is constructor preferred?
- How does Spring resolve multiple beans of the same type?
- `@Primary` vs `@Qualifier`?
- What triggers `NoUniqueBeanDefinitionException`?

Hands-on Exercise

Create two `PaymentGateway` beans; make one `@Primary`, inject the other with `@Qualifier`; then switch to `ObjectProvider` to pick at runtime.

2.6 Circular Dependencies

Core Concept

`A → B → A`. Spring resolves **setter/field** singleton cycles using a **three-level cache** (early exposure of a raw, not-yet-initialized reference):

```
singletonObjects (fully ready)
earlySingletonObjects (raw, exposed early)
singletonFactories (factory to build early ref)
```

Flow: creating A → put A's factory in level-3 → inject B → B needs A → gets A's *early* reference from cache → B finishes → A finishes. Works because the reference exists before initialization completes.

Constructor injection cycles cannot be resolved (the object doesn't exist yet to expose) →

`BeanCurrentlyInCreationException`. Spring Boot 2.6+ also **prohibits circular references by default** (must set `spring.main.allow-circular-references=true`).

ASCII

```
create A -> expose earlyRef(A) -> inject into A: need B
-> create B -> inject earlyRef(A) into B -> B done
-> back to A -> A done
(Constructor cycle: no earlyRef possible -> exception)
```

Real Production Example / Fix

A cycle usually signals a design smell. Fixes: **refactor** (extract a third component), break with `@Lazy` on one dependency (injects a proxy), or use setter injection / `ApplicationEventPublisher`.

Interview Q / Follow-ups

- How does Spring resolve circular deps? Why only setter/field, not constructor?
- Explain the three-level cache.
- How does `@Lazy` break a cycle? (*injects a lazy proxy; real bean resolved on first use.*)
- Why did Spring Boot disable circular refs by default?

2.7 Profiles, Environment & Property Sources

Core Concept

- **Environment** abstracts profiles + properties. Beans/config can be activated per **profile** (`@Profile("prod")`).
- **Property sources** are layered with a defined precedence; `@Value("${...}")` and `Environment.getProperty()` read them.

Spring Boot property precedence (high → low, abridged)

1. Command-line args (`--server.port=9090`)
2. OS environment variables / `SPRING_APPLICATION_JSON`
3. `application-{profile}.properties/yml` (profile-specific)
4. `application.properties/yml` (default)
5. `@PropertySource`, defaults

Env vars map: `SERVER_PORT` → `server.port` (relaxed binding).

Real Production Example

`application-prod.yml` points at the real DB and Kafka cluster; `-Dspring.profiles.active=prod` (or `SPRING_PROFILES_ACTIVE=prod`) selects it. `@Profile("!prod")` beans provide in-memory test doubles locally.

Interview Q / Follow-ups

- How do profiles work; how to activate them?
- Property source precedence — where does an env var rank vs `application.yml`?
- `@Value` vs `@ConfigurationProperties`? (latter is type-safe, grouped, validated, relaxed-bound — preferred for many props.)
- What is relaxed binding?

Hands-on Exercise

Bind a `@ConfigurationProperties(prefix="app.mail")` record; override one field via an environment variable and confirm precedence.

Best Practices

Never hardcode secrets; externalize per environment; prefer `@ConfigurationProperties` for groups; keep profile-specific overrides minimal.

Module 2 — One-Page Cheat Sheet

Concept	Key point
IoC/DI	Container owns object graph; inject via constructor
Context	<code>ApplicationContext</code> = <code>BeanFactory</code> + events/AOP/i18n/eager singletons
Lifecycle	instantiate → inject → aware → BPP.before → <code>@PostConstruct</code> → <code>afterPropertiesSet</code> → BPP.after(proxy) → ready → <code>@PreDestroy</code> → de
Proxy	Created in BPP-after; self-invocation bypasses AOP
Scopes	singleton default; prototype not destroyed by container; use <code>ObjectProvider</code>
Stereotypes	<code>@Repository</code> adds exception translation; <code>@Configuration</code> CGLIB-proxied
<code>@Bean</code> vs <code>@Component</code>	<code>@Bean</code> for 3rd-party types
Autowire	type → <code>@Primary</code> → <code>@Qualifier</code> → name
Injection	constructor (final, testable, fail-fast) preferred
Circular	3-level cache fixes setter/field, not constructor; disabled by default 2.6+
Profiles/Props	layered precedence; cmd-line > env > profile yml > default yml

Module 2 — Top Interview Questions

1. Explain the full bean lifecycle and where AOP proxies are applied.

2. Why does self-invocation break `@Transactional`/`@Async`?
3. Constructor vs field injection — trade-offs.
4. How does Spring resolve circular dependencies (three-level cache)? Constructor limitation?
5. `@Component` vs `@Bean` vs `@Configuration`; why is `@Configuration` proxied?
6. `BeanFactory` vs `ApplicationContext`; `BeanPostProcessor` vs `BeanFactoryPostProcessor`.
7. Bean scopes; injecting prototype into singleton.
8. How does autowiring resolve ambiguity? `@Primary` vs `@Qualifier`.
9. Profiles and property-source precedence.
10. `@Value` vs `@ConfigurationProperties`.

Module 2 — Common Mistakes

- Field injection everywhere (untestable, hidden deps).
- Expecting `@Transactional` to work on self-invoked / private methods.
- Assuming singleton beans are automatically thread-safe.
- Injecting a prototype into a singleton and expecting freshness.
- Business code calling `getBean()` (service-locator anti-pattern).

Module 2 — Mock Interview

1. “`@Transactional` on `save()` isn't rolling back when called from `process()` in the same class. Why?” → self-invocation bypasses the proxy; move `save()` to another bean or call via injected self-proxy.
2. “App won't start: `BeanCurrentlyInCreationException`.” → constructor circular dependency; refactor or `@Lazy`.
3. “Two `DataSource` beans, injection fails.” → `NoUniqueBeanDefinitionException`; add `@Primary`/`@Qualifier`.
4. “How do you supply different configs for dev/staging/prod without rebuilding?” → profiles + externalized `application-{profile}.yaml` + env vars.
5. “Walk me through what happens between JVM start and the context being ready.” → read definitions → BFPPs → instantiate → inject → BPPs (proxies) → `@PostConstruct` → ready.

Next → Module 3: Spring Boot (auto-configuration, starters, actuator).